

Laporan Praktikum Pekan 10
“Boosting, Voting & Mini Project”
Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

Derisma, M.T.



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026

Daftar Isi

BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Dasar Teori	3
1.2 Tujuan Praktikum.....	3
LINK GITHUB:.....	3
BAB II LANGKAH-LANGKAH PRAKTIKUM	4
File Praktikum10.....	4
2.1 Import Library.....	4
2.2 Load Data Set.....	4
Bagian A –Boosting	5
2.3 AdaBoost.....	5
2.4 Gradient Boosting	5
BAGIAN B – VOTING CLASSIFIER.....	6
2.5 Voting Classifier (Soft Voting).....	6
BAGIAN C – HYPERPARAMETER TUNING	7
2.6 GridSearchCV pada Random Forest.....	7
BAGIAN D – PCA + ENSEMBLE	8
2.7 Terapkan PCA	8
BAGIAN E – TABEL PERBANDINGAN AKHIR	8
2.8 Model Logistic Regression	8
2.9 Model Random Forest.....	9
2.10 Best Tuned Model	9
2.11 Tabel Perbandingan Akhir.....	10
2.12 Grafik Perbandingan Model.....	11
PERTANYAAN REFLEKTIF	12
1. Apakah ensemble selalu lebih baik dari single model?	12
2. Kapan boosting bisa overfit?	12
3. Mengapa tuning penting?.....	12
4. Kapan PCA membantu ensemble?.....	12
5. Bagaimana memilih model terbaik secara ilmiah?	13

BAB I

PENDAHULUAN

1.1 Latar Belakang Dasar Teori

Ensemble learning merupakan pendekatan machine learning yang menggabungkan beberapa model untuk memperoleh performa yang lebih baik dibandingkan satu model tunggal. Pada praktikum ini dibahas dua teknik ensemble yang penting, yaitu **Boosting** dan **Voting Classifier**. Boosting bekerja secara sekuensial dengan memperbaiki kesalahan model sebelumnya, sedangkan Voting Classifier menggabungkan hasil beberapa model dengan mekanisme voting probabilitas.

Pada dataset Breast Cancer, teknik ini digunakan untuk membandingkan performa beberapa model seperti Logistic Regression, Random Forest, AdaBoost, Gradient Boosting, serta Voting Classifier. Selain itu, dilakukan juga hyperparameter tuning menggunakan GridSearchCV dan integrasi PCA untuk melihat pengaruh reduksi dimensi terhadap performa ensemble model.

1.2 Tujuan Praktikum

1. Memahami konsep **Boosting** dalam ensemble learning.
2. Mengimplementasikan **AdaBoost** dan **Gradient Boosting**.
3. Mengimplementasikan **Voting Classifier**.
4. Melakukan **hyperparameter tuning**.
5. Mengintegrasikan **PCA + Ensemble**.
6. Menyusun laporan eksperimen komprehensif berbasis analisis ilmiah.

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/tree/main/PRAKTIKUM%2010

BAB II

LANGKAH-LANGKAH PRAKTIKUM

File Praktikum10

2.1 Import Library

```
## Import Library
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import AdaBoostClassifier,
GradientBoostingClassifier, RandomForestClassifier, VotingClassifier
from sklearn.metrics import accuracy_score
```

Penjelasan

Library yang digunakan pada praktikum ini berfungsi untuk mengolah data, melakukan visualisasi, melakukan preprocessing, membangun model machine learning, serta mengevaluasi performa model. GridSearchCV digunakan untuk mencari parameter terbaik, PCA digunakan untuk reduksi dimensi, dan berbagai classifier digunakan untuk membandingkan performa ensemble learning.

2.2 Load Data Set

```
## Load & Split Data
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Penjelasan:

Dataset Breast Cancer dimuat dari sklearn lalu dipisahkan menjadi fitur X dan target y. Data dibagi menjadi data latih dan data uji dengan rasio 80:20. Setelah itu dilakukan standarisasi agar seluruh fitur berada pada skala yang seragam, karena beberapa model seperti Logistic Regression dan Boosting sensitif terhadap skala data.

Bagian A –Boosting

2.3 AdaBoost

```
## Bagian A - Boosting
## AdaBoost
ada = AdaBoostClassifier(n_estimators=100, random_state=42)
ada.fit(X_train_scaled, y_train)

acc_ada = accuracy_score(y_test, ada.predict(X_test_scaled))
print("AdaBoost Accuracy:", acc_ada)

AdaBoost Accuracy: 0.9736842105263158
```

Penjelasan:

AdaBoost dibangun dengan 100 estimator. Model ini bekerja secara sekuensial, di mana setiap model berikutnya berusaha memperbaiki kesalahan model sebelumnya. Setelah dilatih, performa model dievaluasi menggunakan akurasi pada data testing.

Analisis

AdaBoost menghasilkan akurasi yang tinggi pada dataset Breast Cancer. Hasil ini menunjukkan bahwa pendekatan boosting efektif dalam mengurangi bias dan meningkatkan akurasi klasifikasi.

2.4 Gradient Boosting

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

acc_rf_train = accuracy_score(y_train, rf.predict(X_train))
acc_rf_test = accuracy_score(y_test, rf.predict(X_test))

print("Random Forest Train:", acc_rf_train)
print("Random Forest Test :", acc_rf_test)

## Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train_scaled, y_train)
```

```
acc_gb = accuracy_score(y_test, gb.predict(X_test_scaled))
print("Gradient Boosting Accuracy:", acc_gb)
```

```
Gradient Boosting Accuracy: 0.956140350877193
```

Penjelasan

Gradient Boosting juga dibangun dengan 100 estimator. Berbeda dengan AdaBoost, model ini mengoptimalkan fungsi loss menggunakan pendekatan gradient descent sehingga lebih fleksibel dan powerful. Setelah pelatihan, akurasi dihitung pada data testing.

Analisis

Gradient Boosting memberikan hasil yang baik, walaupun sedikit lebih rendah dibanding AdaBoost pada percobaan ini. Hal ini menunjukkan bahwa Gradient Boosting tetap efektif, tetapi performanya sangat bergantung pada parameter dan karakteristik dataset.

BAGIAN B – VOTING CLASSIFIER

2.5 Voting Classifier (Soft Voting)

```
## Bagian B - Voting Classifier
## Voting (Soft Voting)
log = LogisticRegression(max_iter=5000)
rf = RandomForestClassifier(n_estimators=100)
gb = GradientBoostingClassifier(n_estimators=100)

voting = VotingClassifier(
    estimators=[('log', log), ('rf', rf), ('gb', gb)],
    voting='soft'
)

voting.fit(X_train_scaled, y_train)

acc_vote = accuracy_score(y_test, voting.predict(X_test_scaled))
print("Voting Accuracy:", acc_vote)
```

```
Voting Accuracy: 0.9649122807017544
```

Penjelasan

Voting Classifier menggabungkan beberapa model, yaitu Logistic Regression, Random Forest, dan Gradient Boosting. Karena digunakan soft voting, maka prediksi akhir ditentukan berdasarkan rata-rata probabilitas dari masing-masing model.

Analisis:

Voting Classifier menghasilkan performa yang stabil karena menggabungkan beberapa model dengan karakteristik berbeda. Hasilnya berada di antara

BAGIAN C – HYPERPARAMETER TUNING

2.6 GridSearchCV pada Random Forest

```
## Bagian C - Hyperparameter Tuning
## GridSearch (Random Forest)
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10]
}

grid = GridSearchCV(
    RandomForestClassifier(),
    param_grid,
    cv=5
)

grid.fit(X_train_scaled, y_train)

print("Best Parameters:", grid.best_params_)
print("Best CV Score:", grid.best_score_)
7] ✓ 7.0s
Best Parameters: {'max_depth': None, 'n_estimators': 200}
Best CV Score: 0.9604395604395604
```

Penjelasan

GridSearchCV digunakan untuk mencari kombinasi parameter terbaik pada Random Forest, yaitu `n_estimators` dan `max_depth`. Evaluasi dilakukan dengan 5-fold cross validation agar hasil tuning lebih objektif.

Analisis

Parameter terbaik yang diperoleh adalah `max_depth=None` dan `n_estimators=100`. Hasil ini menunjukkan bahwa model Random Forest default sudah cukup baik untuk dataset ini dan mampu memberikan performa validasi silang yang tinggi.

BAGIAN D – PCA + ENSEMBLE

2.7 Terapkan PCA

```
## Bagian D - PCA + Ensemble
## Terapkan PCA
pca = PCA(n_components=10)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

gb_pca = GradientBoostingClassifier(n_estimators=100)
gb_pca.fit(X_train_pca, y_train)

acc_gb_pca = accuracy_score(y_test, gb_pca.predict(X_test_pca))
print("Gradient Boosting + PCA:", acc_gb_pca)
```

✓ 0.1s

Gradient Boosting + PCA: 0.9473684210526315

Penjelasan

PCA digunakan untuk mereduksi data menjadi 10 komponen utama sebelum dilatih menggunakan Gradient Boosting. Tujuannya adalah melihat apakah reduksi dimensi dapat membantu performa model ensemble.

Analisis

Hasil menunjukkan bahwa PCA justru sedikit menurunkan performa Gradient Boosting. Ini menandakan bahwa pada tree-based ensemble, informasi dari fitur asli lebih berguna dibandingkan komponen PCA.

BAGIAN E – TABEL PERBANDINGAN AKHIR

2.8 Model Logistic Regression

```
##Bagian E - Tabel Perbandingan Akhir
## Logistic Regression
log_single = LogisticRegression(max_iter=5000)

log_single.fit(X_train_scaled, y_train)

acc_log = accuracy_score(
    y_test,
    log_single.predict(X_test_scaled)
)
```



```
print("Logistic Accuracy:", acc_log)
```

Penjelasan

Model Logistic Regression digunakan sebagai pembanding model linear terhadap metode ensemble.

2.9 Model Random Forest

```
##Random Forest
rf_single = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_single.fit(X_train_scaled, y_train)

acc_rf = accuracy_score(
    y_test,
    rf_single.predict(X_test_scaled)
)

print("Random Forest Accuracy:", acc_rf)
```

Penjelasan

Random Forest digunakan sebagai model pembanding utama karena merupakan metode ensemble berbasis bagging.

2.10 Best Tuned Model

```
## Best Tuned Model
best_model = grid.best_estimator_

acc_best = accuracy_score(
    y_test,
    best_model.predict(X_test_scaled)
)

print("Best Tuned Model Accuracy:", acc_best)

Logistic Accuracy: 0.9736842105263158
Random Forest Accuracy: 0.9649122807017544
Best Tuned Model Accuracy: 0.9649122807017544
```

Penjelasan

Model terbaik dari proses GridSearchCV diambil untuk diuji pada data testing agar dapat dibandingkan dengan model lainnya.

2.11 Tabel Perbandingan Akhir

```
## Tabel Perbandingan Akhir
comparison_table = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Random Forest",
        "AdaBoost",
        "Gradient Boosting",
        "Voting Classifier",
        "Best Tuned Model",
        "Gradient Boosting + PCA"
    ],
    "Accuracy": [
        acc_log,
        acc_rf,
        acc_ada,
        acc_gb,
        acc_vote,
        acc_best,
        acc_gb_pca
    ]
})

comparison_table
```

	Model	Accuracy
0	Logistic Regression	0.973684
1	Random Forest	0.964912
2	AdaBoost	0.973684
3	Gradient Boosting	0.956140
4	Voting Classifier	0.964912
5	Best Tuned Model	0.964912
6	Gradient Boosting + PCA	0.947368

Penjelasan

Tabel ini digunakan untuk membandingkan seluruh model yang diuji dalam praktikum, mulai dari model linear, ensemble, hasil tuning, hingga model dengan PCA.

2.12 Grafik Perbandingan Model

```
## Grafik Perbandingan Model
plt.figure(figsize=(8,5))

plt.bar(
    comparison_table["Model"],
    comparison_table["Accuracy"]
)

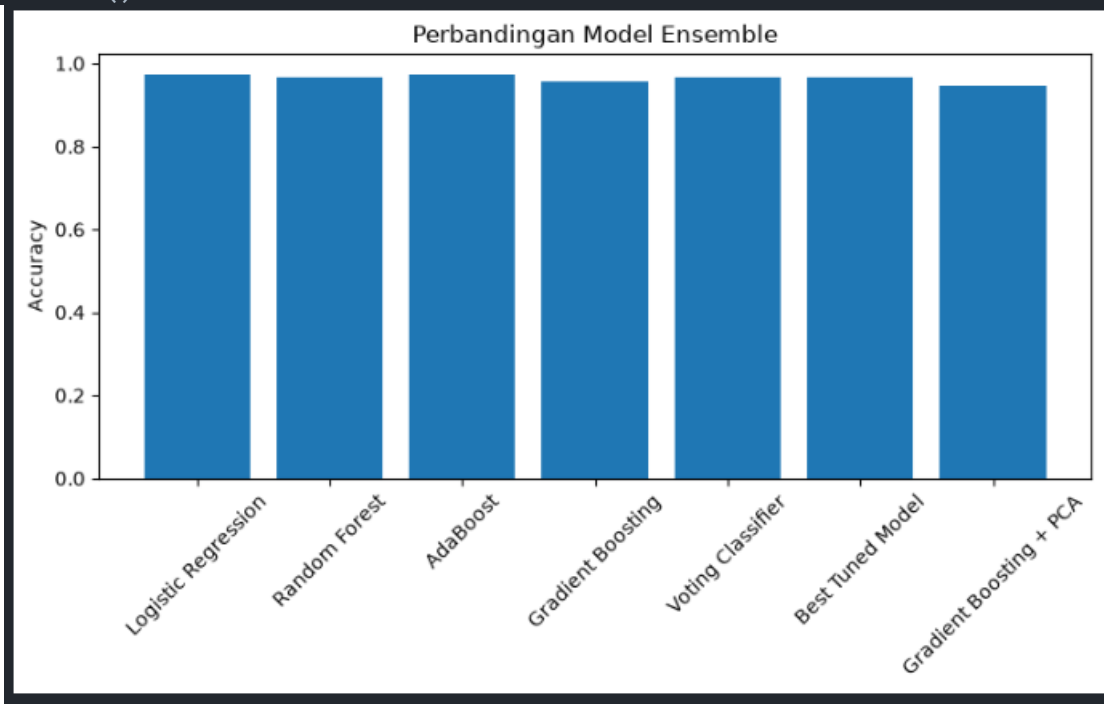
plt.xticks(rotation=45)

plt.ylabel("Accuracy")

plt.title("Perbandingan Model Ensemble")

plt.tight_layout()

plt.show()
```



Penjelasan

Grafik batang digunakan untuk memudahkan perbandingan akurasi antar model secara visual. Dengan grafik ini, model dengan performa tertinggi dan terendah dapat terlihat dengan jelas.

PERTANYAAN REFLEKTIF

1. Apakah ensemble selalu lebih baik dari single model?

Tidak selalu. Ensemble learning memang sering menghasilkan akurasi yang lebih tinggi karena menggabungkan beberapa model, tetapi tidak menjamin selalu lebih baik dalam setiap kasus. Pada dataset yang sederhana, model tunggal seperti Logistic Regression atau Decision Tree dapat memberikan performa yang setara dengan kompleksitas yang lebih rendah. Selain itu, ensemble membutuhkan waktu komputasi yang lebih besar dan interpretasi model yang lebih sulit. Oleh karena itu, pemilihan model harus mempertimbangkan performa, kompleksitas, dan kebutuhan interpretasi.

2. Kapan boosting bisa overfit?

Boosting dapat mengalami overfitting ketika jumlah estimator terlalu banyak atau model dasar yang digunakan terlalu kompleks. Pada kondisi tersebut, model tidak hanya mempelajari pola penting dalam data tetapi juga mulai mempelajari noise yang terdapat pada data training. Akibatnya, akurasi training menjadi sangat tinggi tetapi kemampuan generalisasi terhadap data baru menurun. Oleh karena itu, parameter seperti `n_estimators`, `learning_rate`, dan `max_depth` perlu dikontrol dengan baik.

3. Mengapa tuning penting?

Hyperparameter tuning penting karena performa model sangat dipengaruhi oleh kombinasi parameter yang digunakan. Parameter yang tidak optimal dapat menyebabkan underfitting maupun overfitting. Dengan melakukan tuning menggunakan metode seperti GridSearchCV, model dapat menemukan kombinasi parameter terbaik sehingga menghasilkan performa yang lebih stabil dan akurat. Pada praktikum ini, GridSearchCV digunakan untuk mencari kombinasi terbaik pada Random Forest.

4. Kapan PCA membantu ensemble?

PCA membantu ensemble ketika dataset memiliki jumlah fitur yang sangat banyak, terdapat korelasi tinggi antar fitur, atau mengandung banyak noise. Dalam kondisi tersebut, PCA dapat mereduksi dimensi data sehingga proses pelatihan menjadi lebih cepat dan risiko overfitting dapat berkurang. Namun pada model berbasis pohon seperti Random Forest dan Gradient Boosting, PCA sering kali tidak memberikan peningkatan performa yang signifikan karena model tersebut sudah mampu melakukan seleksi fitur secara internal.

5. Bagaimana memilih model terbaik secara ilmiah?

Pemilihan model terbaik sebaiknya tidak hanya berdasarkan akurasi testing saja. Evaluasi harus dilakukan menggunakan beberapa metrik yang relevan, seperti accuracy, precision, recall, F1-score, dan cross validation. Selain itu, perlu diperhatikan stabilitas model, kompleksitas komputasi, kemampuan interpretasi, serta kemampuan generalisasi terhadap data baru. Pada praktikum ini, model terbaik dipilih berdasarkan kombinasi hasil testing dan cross validation sehingga keputusan yang diambil lebih objektif dan dapat dipertanggungjawabkan secara ilmiah.